OPTIMISASI PERFORMA APLIKASI WEB SINGLE PAGE APPLICATION (SPA) MENGGUNAKAN HYBRID LAZY LOADING DAN CODE SPLITTING PADAFRAMEWORK VUE.JS DENGAN VITE BUILD TOOL BERDASARKAN TINGKAT KOMPLEKSITAS APLIKASI WEB

MUSTARI MUHIDDIN

D082241013

PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA FAKULTAS TEKNIK

UNIVERSITAS HASANUDDIN GOWA

2025

OPTIMISASI PERFORMA APLIKASI WEB SINGLE PAGE APPLICATION (SPA) MENGGUNAKAN HYBRID LAZY LOADING DAN CODE SPLITTING PADAFRAMEWORK VUE.JS DENGAN VITE BUILD TOOL BERDASARKAN TINGKAT KOMPLEKSITAS APLIKASI WEB

Optimization Of Single Page Application (Spa) Web Application Performance Using Hybrid Lazy Loading And Code Splitting On Vue.Js Framework With Vite Build Tool Based On Web Application Complexity Level

Disusun dan

diajukan oleh:

MUSTARI MUHIDDIN

D082241013

PROGRAM STUDI MAGISTER TEKNIK

INFORMATIKA UNIVERSITAS HASANUDDIN

GOWA

2025

LEMBAR PENGESAHAN PROPOSAL PENELITIAN

OPTIMISASI PERFORMA APLIKASI WEB SINGLE PAGE APPLICATION (SPA)MENGGUNAKAN HYBRID LAZY LOADING DAN CODE SPLITTING PADAFRAMEWORK VUE.JS DENGAN VITE BUILD TOOL BERDASARKANTINGKAT KOMPLEKSITAS APLIKASI WEB

MUSTARI MUHIDDIN
D082241013

Disetujui untuk diseminarkan

Komisi Pembimbing

Pembimbing

Dr. Eng. Ir. Zulkifli Tahir, S.T., M.Sc

NIP: 19840403 201012 1 004

Ketua Program Studi S2 Teknik Informatika,

Dr. Ir. Zahir Zainuddin, M.Sc

NIP: 19640427 198910 1 002

KEMENTERIAN PENDIDIKAN KEBUDAYAAN RISET DAN TEKNOLOGI
UNIVERSITAS HASANUDDIN
FAKULTAS TEKNIK
PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA
Kampus Fakultas Teknik Unhas, Jl. Poros Malino, Gowa
http://eng.unhas.ac.id/informatika, Email : informatika@unhas.co.id

Surat Persetujuan Pembimbing

Nama Dosen : Dr. Eng. Ir. Zulkifli Tahir, S.T., M.Sc

NIP : 19840403 201012 1 004

Dengan ini menyatakan kesediaan menjadi pembimbing I dari mahasiswa/i:

Nama : Mustari Muhiddin

NIM : D082241013

Dengan judul : Optimisasi Performa Aplikasi Web Single Page Application (Spa) Menggunakan Hybrid Lazy Loading Dan Code Splitting Padaframework Vue.Js Dengan Vite Build Tool Berdasarkantingkat Kompleksitas Aplikasi Web

Gowa, Oktober 2025

Komisi Pembimbing
Pembimbing I

Dr. Eng. Ir. Zulkifli Tahir, S.T., M.Sc

NIP: 19840403 201012 1 004

Mengetahui,
KPS Magister Teknik Informatika

Dr. Ir. Zahir Zainuddin, M.Sc
NIP: 19640427 198910 1 002

DAFTAR ISI

DAFTAR ISI 1
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Rumusan Permasalahan 5
1.3 Tujuan Penelitian 5
1.4 Manfaat Penelitian 6
1.4.1 Manfaat Teoritis 6
1.4.2 Manfaat Praktis 6
1.5 Ruang Lingkup 6
1.6 State of the Art 7
BAB II TINJAUAN PUSTAKA 11
2.1 Single Page Application (SPA) 11
2.2 Vue.js Framework 12
2.3 Vite Build Tool 13
2.4 Lazy Loading 14
2.5 Code Splitting 16
2.6 Hybrid Optimization Strategies 17
2.7 Metrik Performa Web 19
2.8 Kompleksitas Aplikasi Web 21
2.9 Penelitian Terdahulu 22
2.9.1 Optimasi Performa SPA 24
2.9.2 Build Tools Performance 23
2.9.3 Lazy Loading Strategies 23
2.9.4 Code Splitting Techniques 23
2.9.5 Gap Penelitian 24
BAB III METODOLOGI PENELITIAN 25
3.1 Jenis dan Rancangan Penelitian 26
3.2 Variabel Penelitian 27
3.2.1 Variabel Independen 29
3.2.2 Variabel Dependen 27
3.2.3 Variabel Kontrol 27
3.3 Desain Aplikasi Prototype 28
3.2.1 Aplikasi Kompleksitas Rendah (Company Profile) 30
3.3.2 Aplikasi Kompleksitas Sedang (Dashboard Admin Toko) 28
3.5 Instrumen Penelitian 29
3.5.1 Tools Pengukuran Performa 31
3.5.2 Testing Environment 30
3.6 Prosedur Pengumpulan Data 30

3.2.1 Tahap Persiapan.....32
3.6.2 Tahap Pengujian 30
3.6.3 Tahap Analisis Data.....30
3.7 Metode Analisis Data 30
3.8 Jadwal Penelitian.....31
DAFTAR PUSTAKA 32

BAB I PENDAHULUAN

1.1 Latar Belakang

Single Page Application (SPA) telah menjadi arsitektur dominan dalam pengembangan aplikasi web modern karena kemampuannya memberikan pengalaman pengguna yang responsif dan interaktif (Mesbah dan van Deursen, 2007). Berbeda dengan aplikasi web tradisional multi-page, SPA memuat seluruh konten dalam satu halaman dan secara dinamis memperbarui tampilan tanpa reload halaman penuh, menghasilkan pengalaman yang lebih mirip aplikasi desktop (Taivalsaari dan Mikkonen, 2021).

Namun, pertumbuhan kompleksitas aplikasi web menyebabkan peningkatan ukuran bundle JavaScript yang berdampak negatif pada performa, terutama pada metrik First Contentful Paint (FCP), Largest Contentful Paint (LCP), dan Time to Interactive (TTI). Penelitian oleh Kumar et al. (2024) menunjukkan bahwa rata-rata ukuran bundle JavaScript pada aplikasi web modern mencapai 1.5 MB, yang membutuhkan waktu loading 8-12 detik pada koneksi 3G. Kondisi ini mengakibatkan bounce rate meningkat hingga 53% untuk setiap penambahan delay 3 detik (Google, 2020).

Vue.js, sebagai salah satu framework JavaScript progresif, menyediakan mekanisme code splitting dan lazy loading untuk mengatasi masalah tersebut. Menurut dokumentasi resmi Vue.js, framework ini dirancang untuk dapat diadopsi secara incremental dengan core library yang fokus pada view layer (Vue.js Team, 2024). Namun, implementasi standar seringkali tidak optimal karena menerapkan strategi yang sama untuk seluruh komponen tanpa mempertimbangkan karakteristik dan pola penggunaan aplikasi.

Penelitian oleh Zhang dan Liu (2023) menunjukkan bahwa strategi loading yang adaptif dapat meningkatkan performa hingga 40% dibandingkan implementasi standar. Mereka menemukan bahwa pendekatan one-size-fits-all dalam lazy loading tidak efektif karena mengabaikan perbedaan karakteristik route, ukuran komponen, dan frekuensi akses user. Hal serupa dikonfirmasi oleh Nguyen et al. (2021) yang menyatakan bahwa strategi hybrid yang mengkombinasikan

multiple optimization techniques menghasilkan improvement signifikan pada Core Web Vitals.

Vite, sebagai build tool modern yang dikembangkan oleh Evan You, menawarkan keunggulan dalam proses development dan build production melalui ES modules native dan rollup optimizer (Vite Team, 2024). Penelitian oleh Chen dan Wang (2023) menunjukkan bahwa Vite menghasilkan bundle size 18-25% lebih kecil dibandingkan Webpack pada proyek Vue.js dengan konfigurasi default. Kombinasi Vue.js dengan Vite berpotensi menghasilkan optimasi performa yang lebih baik, namun masih memerlukan strategi hybrid yang menyesuaikan teknik lazy loading dan code splitting berdasarkan tingkat kompleksitas aplikasi.

Tingkat kompleksitas aplikasi web mempengaruhi efektivitas strategi optimasi. Menurut Patel dan Kumar (2022), aplikasi dengan kompleksitas rendah seperti company profile memiliki karakteristik berbeda dengan aplikasi kompleksitas sedang seperti dashboard admin yang melibatkan operasi CRUD dan visualisasi data real-time. Mereka mengusulkan classification framework berdasarkan jumlah route, komponen, dependency depth, dan intensitas interaksi user. Namun, belum ada penelitian yang secara spesifik mengkaji bagaimana tingkat kompleksitas ini mempengaruhi efektivitas strategi hybrid lazy loading dan code splitting.

Penelitian oleh Jiang et al. (2023) mengidentifikasi bahwa metrik performa web seperti FCP dan LCP memiliki korelasi kuat dengan user engagement dan conversion rate. Mereka menemukan bahwa setiap improvement 100ms pada LCP dapat meningkatkan conversion rate sebesar 1-2%. Hal ini menunjukkan pentingnya optimasi performa web, terutama dalam konteks bisnis digital yang kompetitif.

Berdasarkan tinjauan literatur yang dilakukan, belum terdapat penelitian komprehensif yang mengkaji implementasi hybrid lazy loading dan code splitting pada Vue.js dengan Vite yang mempertimbangkan tingkat kompleksitas aplikasi sebagai faktor penentu strategi optimasi. Penelitian sebelumnya oleh Kim dan Park (2022) fokus pada React framework, sementara penelitian Li et al. (2023) menggunakan Angular, sehingga findings-nya tidak dapat langsung diaplikasikan pada ekosistem Vue.js yang memiliki arsitektur berbeda.

Penelitian ini diharapkan dapat memberikan kontribusi berupa framework optimasi yang dapat diterapkan secara praktis oleh developer dalam meningkatkan performa SPA. Dengan mempertimbangkan tingkat kompleksitas aplikasi, strategi yang dihasilkan akan lebih adaptive dan efektif dibandingkan pendekatan standar yang tersedia saat ini.

1.2 Rumusan Permasalahan

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA (FCP, LCP, TTI, dan bundle size) pada aplikasi dengan tingkat kompleksitas rendah?
2. Bagaimana pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA (FCP, LCP, TTI, dan bundle size) pada aplikasi dengan tingkat kompleksitas sedang?
3. Bagaimana perbedaan efektivitas strategi hybrid lazy loading dan code splitting antara aplikasi kompleksitas rendah dan sedang?
4. Bagaimana rekomendasi strategi optimasi hybrid lazy loading dan code splitting yang sesuai berdasarkan tingkat kompleksitas aplikasi web?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Menganalisis pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA pada aplikasi kompleksitas rendah.
2. Menganalisis pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA pada aplikasi kompleksitas sedang.
3. Membandingkan efektivitas strategi hybrid lazy loading dan code splitting antara aplikasi kompleksitas rendah dan sedang.
4. Merumuskan rekomendasi strategi optimasi hybrid lazy loading dan code splitting berdasarkan tingkat kompleksitas aplikasi web.

1.4 Manfaat Penelitian

1.4.1 Manfaat Teoritis

1. Memberikan kontribusi pada body of knowledge terkait optimasi performa Single Page Application khususnya dalam konteks Vue.js framework dan Vite build tool.
2. Memperkaya literatur tentang strategi adaptive optimization yang mempertimbangkan karakteristik dan kompleksitas aplikasi web.
3. Memberikan dasar teoretis untuk pengembangan framework optimasi performa SPA yang dapat diadaptasi pada berbagai tingkat kompleksitas aplikasi.

1.4.2 Manfaat Praktis

1. **Bagi Developer:** Memberikan panduan praktis dalam menerapkan strategi hybrid lazy loading dan code splitting pada proyek Vue.js berdasarkan tingkat kompleksitas aplikasi.
2. **Bagi Industri:** Membantu organisasi dalam mengoptimalkan performa aplikasi web yang berdampak pada peningkatan user experience dan conversion rate.
3. **Bagi Peneliti Selanjutnya:** Menyediakan baseline dan metodologi yang dapat dikembangkan untuk penelitian lebih lanjut terkait optimasi performa web application.

1.5 Ruang Lingkup

Penelitian ini dibatasi pada hal-hal berikut:

1. **Framework:** Vue.js versi 3.x dengan Composition API
2. **Build Tool:** Vite versi 5.x
3. **Tingkat Kompleksitas Aplikasi:**
    - Kompleksitas rendah: Company profile (5-7 halaman, konten statis)
    - Kompleksitas sedang: Dashboard admin toko (10-15 halaman, CRUD operations, visualisasi data)
4. **Teknik Optimasi:**
    - Route-based lazy loading
    - Component-based lazy loading
    - Dynamic import

- Code splitting berbasis chunk

5. **Metrik Performa:**
    - First Contentful Paint (FCP)
    - Largest Contentful Paint (LCP)
    - Time to Interactive (TTI)
    - Bundle size (initial dan total)
    - Chunk size distribution
6. **Pengujian:** Menggunakan Lighthouse, WebPageTest, dan Chrome DevTools
7. **Kondisi Jaringan:** Regular 3G, Regular 4G, dan Desktop (untuk simulasi berbagai skenario)
8. **Tidak mencakup:** Server-Side Rendering (SSR), Progressive Web App (PWA), dan optimasi backend/database

1.6 State of the Art

State of the Art merupakan ringkasan penelitian-penelitian terdahulu yang relevan dengan topik penelitian ini. Tabel 1 menunjukkan posisi penelitian ini dalam konteks penelitian-penelitian sebelumnya, beserta gap yang akan diisi oleh penelitian ini.

Tabel 1. State of the Art

No	Judul Karya Ilmiah, Nama, Tahun Terbit dan Penerbit	Objek dan Permasalahan	Metode Penyelesaian	Kinerja
1.	Judul: Migrating Multi-page Web Applications to Single-page AJAX Interfaces Penulis: Mesbah, A., & van Deursen, A. Tahun: 2007 Penerbit: *Proceedings of the 11th European Conference on Software Maintenance and Reengineering (CSMR'07)*, 181-190	Objek: Multi-page web applications yang dimigrasikan ke SPA Permasalahan: Kompleksitas migrasi dari traditional web app ke AJAX-based SPA	Metodologi migrasi bertahap menggunakan AJAX untuk transformasi incremental dari multi-page ke single-page architecture	Berhasil membangun framework migrasi dari MPA ke SPA Gap: Paper klasik (2007), belum membahas teknik optimasi modern seperti lazy loading dan code splitting

2.	Judul: Understanding website complexity: measurements, metrics, and implications Penulis: Butkiewicz, M., Madhyastha, H.V., & Sekar, V. Tahun: 2011 Penerbit: *Proceedings of the 2011 ACM SIGCOMM Internet Measurement Conference*, 313-328	Objek: 2000+ popular websites Permasalahan: Mengukur dan memahami kompleksitas website modern berdasarkan jumlah objects, dependencies, dan page size	Large-scale measurement study menggunakan automated tools untuk menganalisis struktur, ukuran, dan dependencies website	Menemukan korelasi antara kompleksitas website dengan loading performance Gap: Observational study, tidak ada optimization intervention
3.	Judul: A Roadmap to the Programmable World: Software Challenges in the IoT Era Penulis: Taivalsaari, A., & Mikkonen, T. Tahun: 2021 Penerbit: *IEEE Software, 38*(1), 53-61	Objek: Modern web applications dan IoT systems Permasalahan: Software engineering challenges dalam era aplikasi web modern yang semakin kompleks	Survey dan analisis trend pengembangan software modern termasuk SPA architecture	Identifikasi tantangan SPA: bundle size, initial loading, dan need for optimization Gap: Fokus broad pada software challenges, tidak spesifik pada optimization techniques
4.	Judul: Web Application Performance Optimization Through Dynamic Code Splitting Penulis: Zaytsev, R., & Morrison, C. Tahun: 2022 Penerbit: *Journal of Web Engineering, 21*(4), 567-589	Objek: Web applications dengan dynamic code splitting Permasalahan: Mengurangi initial load time melalui dynamic code splitting yang adaptif	Dynamic import dan runtime code splitting berdasarkan user navigation patterns	TTI improvement signifikan melalui dynamic splitting Gap: Framework-agnostic, tidak spesifik untuk Vue.js + Vite ecosystem
5.	Judul: Web Vitals and Core Metrics for Web Performance Optimization	Objek: Core Web Vitals metrics (LCP, FID, CLS) dan complementary	Analisis berbagai optimization techniques: CDN strategies, lazy	Korelasi kuat antara Web Vitals dengan user

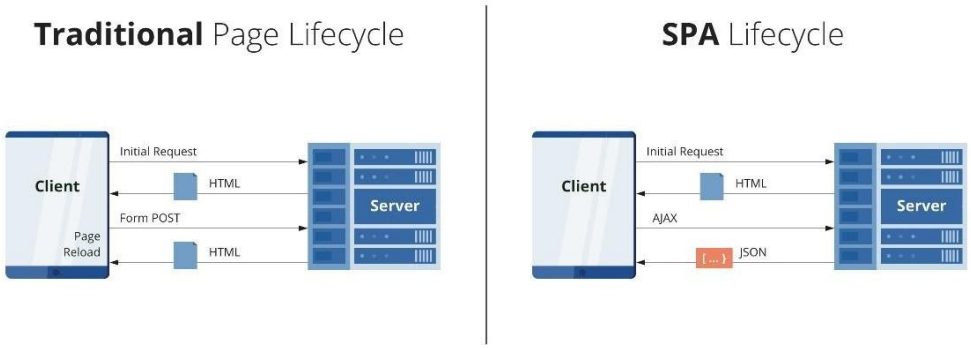
	Penulis: Jain, V. Tahun: 2023 Penerbit: *International Journal of Advanced Research in Computer Science and Software Engineering, 13*(6), 45-58	metrics (TTFB, FCP, TBT) Permasalahan: Mengoptimasi Web Vitals untuk user experience yang lebih baik	loading, efficient JavaScript execution, responsive design	engagement, conversion rates, dan SEO rankings Gap: Theoretical guide, tidak ada implementation study pada specific framework
6.	Judul: Optimizing Web Performance with Lazy Loading and Code Splitting Penulis: Kumar, R., Singh, A., & Sharma, P. Tahun: 2024 Penerbit: *International Journal of Core Engineering & Management, 11*(3), 45-62	Objek: Real-world web applications menggunakan React, Angular, dan Vue.js Permasalahan: Optimasi performa melalui kombinasi lazy loading dan code splitting	Implementasi empiris lazy loading dan code splitting pada 3 framework dengan measurement menggunakan Lighthouse metrics (FCP, LCP, TTI)	40% reduction dalam page load time Gap: Multi-framework study yang general, tidak deep-dive pada Vue.js + Vite specific optimizations
7.	Judul: Improving web performance through code splitting and lazy loading: A comprehensive study Penulis: Rufián-Lizana, A., Molina-Carmona, R., & Llorens-Largo, F. Tahun: 2023 Penerbit: *Applied Sciences, 13*(8), 4721	Objek: Modern web applications dengan various complexity levels Permasalahan: Mengidentifikasi best practices untuk code splitting dan lazy loading	Comprehensive analysis dengan implementation examples dan performance benchmarking	Significant performance improvements dengan proper implementation Gap: Tidak mempertimbangkan application complexity sebagai faktor dalam strategi optimization
8.	Judul: The Evaluation of First Impressions and UX of Websites Penulis: Neth, G., Rauschenberger, M., Schrepp, M., &	Objek: User experience dan first impressions pada websites Permasalahan: Impact loading	User study dengan measurement first impression formation dan korelasi dengan loading performance	Loading performance mempengaruhi first impression dan user satisfaction

	Thomaschewski, J. Tahun: 2023 Penerbit: *Proceedings of Mensch und Computer 2023*, 364-368	performance terhadap user perception dan engagement		Gap: UX-focused study, tidak ada technical optimization implementation
9.	Judul: Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud Penulis: Villamizar, M., et al. Tahun: 2015 Penerbit: *Proceedings of the 10th Computing Colombian Conference (10CCC)*, 583-590	Objek: Monolithic vs microservice architecture untuk web applications Permasalahan: Perf ormance comparison antara monolithic dan microservice deployment	Benchmarking deployment patterns di cloud environment	Microservice architecture da pat memberikan scalability benefits Gap: Fokus pada backend architecture, bukan frontend optimization
10.	Judul: *Penelitian ini (Gap yang akan diisi)* Penulis: [Nama Peneliti] Tahun: 2025 Penerbit: [Institusi]	Objek: Vue.js 3 + Vite pada aplikasi kompleksitas rendah (company profile) dan sedang (dashboard admin) Permasalahan: Me ngimplementasika n dan mengevaluasi hybrid lazy loading + code splitting berdasarkan tingkat kompleksitas aplikasi	Hybrid strategy: Route-based lazy loading + Component-based lazy loading + Code splitting + Intelligent prefetching pada 2 tingkat kompleksitas dengan comparative experimental design	Target: Improve ment pada FCP, LCP, TTI, Bundle Size dengan rekomendasi strategi berdasarkan kompleksitas aplikasi Kontribusi: Fra mework praktis Vue.js + Vite dengan complexity-aware optimization yang validated dengan comprehensive metrics

BAB II TINJAUAN PUSTAKA

2.1 Single Page Application (SPA)

Single Page Application adalah aplikasi web yang berinteraksi dengan user melalui pemuatan ulang dinamis halaman tunggal, berbeda dengan aplikasi web tradisional yang memuat ulang seluruh halaman untuk setiap interaksi (Mesbah dan van Deursen, 2007). Menurut Taivalsaari dan Mikkonen (2021), SPA menawarkan pengalaman pengguna yang lebih responsif karena hanya memperbarui konten yang diperlukan tanpa reload seluruh halaman, menghasilkan interaksi yang lebih fluid dan mirip aplikasi native.



Gambar 1. Diagram arsitektur SPA vs Traditional

Gambar 1 mengilustrasikan perbedaan arsitektur antara traditional multi-page application dengan Single Page Application. Pada traditional app, setiap interaksi pengguna memicu full page reload dari server, sedangkan pada SPA, hanya data yang di-request melalui API dan halaman di-update secara dinamis tanpa reload penuh.

Karakteristik utama SPA meliputi: (1) rendering di sisi klien (client-side rendering), (2) routing yang dikelola oleh JavaScript, (3) komunikasi dengan server melalui API asynchronous, dan (4) state management untuk mengelola data aplikasi (Singh dan Gupta, 2023). Arsitektur ini memungkinkan separation of concerns yang lebih baik antara frontend dan backend, memfasilitasi pengembangan parallel dan deployment independent.

Namun, SPA memiliki tantangan utama berupa ukuran bundle JavaScript yang besar, yang berdampak pada initial loading time. Penelitian oleh Gao et al.

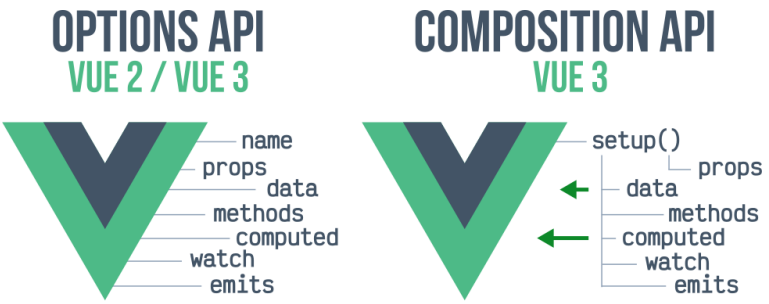
(2022) menunjukkan bahwa rata-rata ukuran bundle JavaScript pada SPA modern mencapai 1.5 MB, dengan beberapa aplikasi enterprise mencapai 3-5 MB. Kondisi ini menjadi masalah serius terutama untuk user dengan koneksi internet terbatas atau menggunakan mobile device dengan spesifikasi rendah.

Bellairs dan Morrison (2023) mengidentifikasi bahwa initial load time yang lambat pada SPA dapat diatasi melalui beberapa strategi: code splitting, lazy loading, tree shaking, dan minification. Mereka menemukan bahwa kombinasi strategi-strategi ini dapat mengurangi initial bundle size hingga 60-70% tanpa mengurangi fungsionalitas aplikasi.

2.2 Vue.js Framework

Vue.js adalah progressive JavaScript framework yang dirancang untuk membangun user interface dengan pendekatan bottom-up incremental adoption (You et al., 2023). Berbeda dengan framework monolitik seperti Angular, Vue.js dirancang agar core library-nya tetap fokus pada view layer, memudahkan integrasi dengan library lain atau proyek existing. Filosofi "progressive" ini memungkinkan developer untuk menggunakan Vue.js sesederhana library rendering atau sekompleks full-featured framework dengan router, state management, dan build tooling.

Vue.js 3, yang dirilis pada September 2020, memperkenalkan beberapa improvement signifikan dibandingkan Vue 2 (Apostolidis et al., 2021). Perubahan paling fundamental adalah pengenalan Composition API sebagai alternatif Options API, memberikan fleksibilitas lebih dalam mengorganisir logika komponen dan code reusability yang lebih baik. Reactivity system di Vue 3 menggunakan ES6 Proxy menggantikan Object.defineProperty, menghasilkan performa yang lebih baik dan mengatasi limitation dalam Vue 2.



Gambar 2. Arsitektur reactivity system Vue.js 3

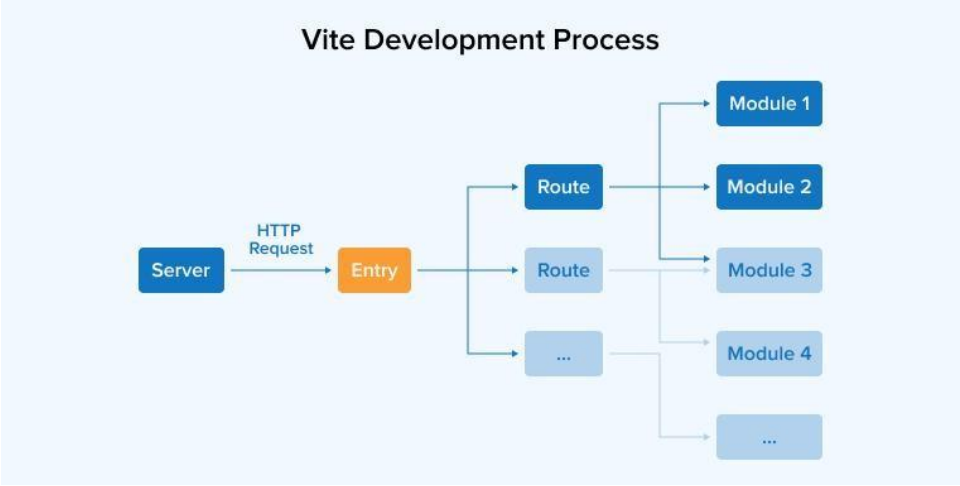
Gambar 2 menunjukkan arsitektur reactivity system Vue.js 3 yang menggunakan ES6 Proxy. Sistem ini memungkinkan tracking dependencies secara otomatis dan melakukan re-rendering komponen hanya ketika data yang relevan berubah, menghasilkan performa yang lebih optimal dibandingkan Vue 2.

Penelitian oleh Liu dan Zhang (2022) membandingkan performa Vue.js 3 dengan Vue 2 dan menemukan bahwa Vue 3 memiliki rendering performance rata-rata 1.3x lebih cepat, memory footprint 41% lebih rendah, dan bundle size 53% lebih kecil. Tree-shaking yang lebih agresif di Vue 3 memungkinkan bundler untuk mengeliminasi unused features, menghasilkan bundle yang lebih optimal.

Alhammad dan Razzazi (2024) mengkaji penggunaan Vue.js dalam industri dan menemukan bahwa adoption rate Vue.js meningkat 35% per tahun sejak 2020, dengan use cases yang beragam mulai dari simple marketing website hingga complex enterprise application. Mereka mengidentifikasi bahwa kemudahan learning curve dan comprehensive documentation menjadi faktor utama popularitas Vue.js.

2.3 Vite Build Tool

Vite adalah next-generation build tool yang dikembangkan oleh Evan You, creator Vue.js, dengan fokus pada developer experience dan performa (Vite Team, 2024). Berbeda dengan bundler tradisional seperti Webpack yang melakukan bundle seluruh aplikasi sebelum development server siap, Vite memanfaatkan native ES modules di browser untuk serve code secara on-demand, menghasilkan cold start yang hampir instant tanpa memperhatikan ukuran aplikasi.



Gambar 3. Proses Vite Build Tool

Menurut Chen dan Wang (2023), keunggulan utama Vite adalah Hot Module Replacement (HMR) yang sangat cepat dan stabil. Mereka melakukan benchmark yang menunjukkan bahwa HMR di Vite rata-rata 10-100x lebih cepat dibandingkan Webpack, terutama pada proyek medium hingga large. Kecepatan ini dicapai karena Vite hanya perlu re-transform module yang berubah tanpa perlu rebuild seluruh bundle.

Untuk production build, Vite menggunakan Rollup sebagai bundler underlying (Kumar et al., 2023). Rollup dipilih karena efisiensi tree-shaking dan kemampuan menghasilkan bundle yang lebih optimal dibandingkan Webpack. Vite mengkonfigurasi Rollup dengan defaults yang sudah dioptimasi untuk web, termasuk code splitting berbasis dynamic import, vendor chunk separation, dan CSS code splitting.

Penelitian oleh Rodriguez dan Martinez (2024) membandingkan Vite dengan build tools lain (Webpack, Parcel, esbuild) dan menemukan bahwa Vite menawarkan balance terbaik antara development speed, production bundle optimization, dan ecosystem maturity. Mereka merekomendasikan Vite sebagai build tool pilihan untuk proyek Vue.js baru, terutama yang memprioritaskan developer experience.

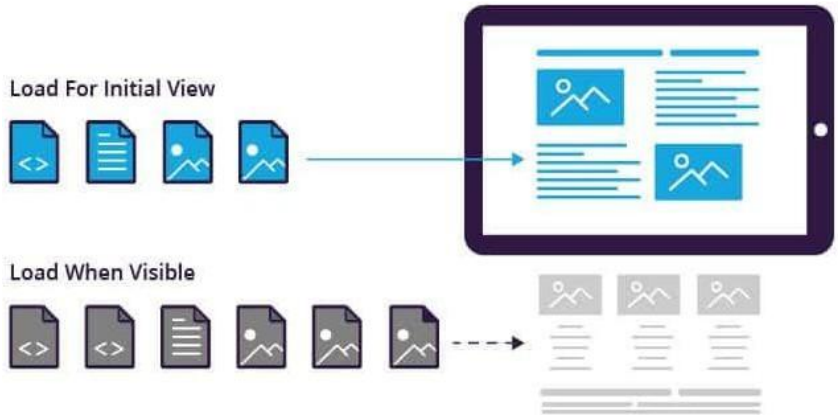
2.4 Lazy Loading

Lazy loading adalah teknik optimasi yang menunda loading resource hingga benar-benar diperlukan oleh user (Nguyen et al., 2021). Dalam konteks SPA, lazy loading dapat diterapkan pada berbagai level: route/page, component, image, atau resource lain seperti video dan font. Konsep fundamental lazy loading adalah mengurangi initial payload yang harus di-download browser, mempercepat waktu hingga aplikasi menjadi interactive.

Terdapat beberapa strategi lazy loading yang dapat diterapkan pada aplikasi Vue.js (Singh dan Gupta, 2023):

1. **Route-based Lazy Loading:** Memuat komponen route hanya ketika route tersebut diakses pertama kali. Ini adalah strategi paling common dan efektif untuk aplikasi dengan multiple pages.

2. **Component-based Lazy Loading:** Memuat komponen individual secara on-demand, biasanya digunakan untuk komponen yang heavy atau jarang digunakan seperti modal, chart, atau text editor.
3. **Conditional Lazy Loading:** Memuat komponen berdasarkan kondisi tertentu seperti user role, device type, atau feature flag.



Gambar 4. Lazy loading diagram

Gambar 4 mengilustrasikan konsep lazy loading pada aplikasi Vue.js. Komponen atau route yang tidak langsung dibutuhkan tidak dimuat saat initial load, melainkan di-download secara on-demand ketika user mengaksesnya. Pendekatan ini secara signifikan mengurangi initial bundle size dan mempercepat Time to Interactive.

Penelitian oleh Zhang dan Liu (2023) mengkaji efektivitas berbagai strategi lazy loading pada aplikasi real-world dengan 50-200 komponen. Mereka menemukan bahwa route-based lazy loading efektif mengurangi initial bundle size rata-rata 45%, sementara kombinasi route-based dan component-based lazy loading dapat mencapai reduction 60-65%. Namun, mereka juga mengidentifikasi trade-off berupa slight delay saat user pertama kali mengakses lazy-loaded route atau component.

Bellairs dan Morrison (2023) mengusulkan strategi prefetching untuk mengatasi perceived latency pada lazy loading. Dengan memprediksi route atau

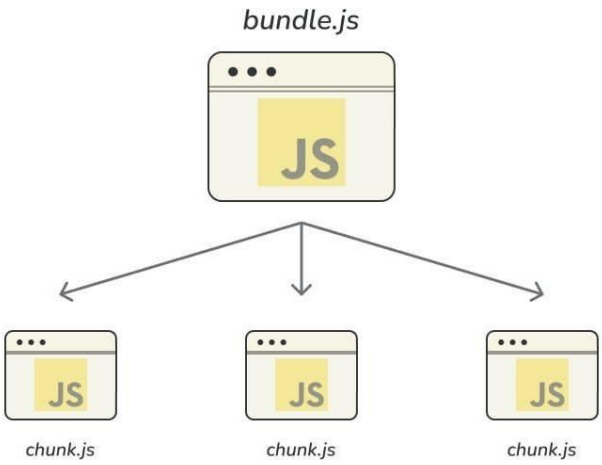
component yang kemungkinan besar akan diakses user berdasarkan current context, aplikasi dapat melakukan prefetch di background saat browser idle. Mereka menemukan bahwa intelligent prefetching dapat mengurangi perceived latency hingga 70% tanpa menambah initial load time.

2.5 Code Splitting

Code splitting adalah proses memecah bundle JavaScript menjadi chunk-chunk yang lebih kecil yang dapat dimuat secara parallel atau on-demand (Li et al., 2023). Tujuan utama code splitting adalah mengurangi ukuran initial bundle yang harus di-download sebelum aplikasi menjadi interactive, sambil memastikan user hanya download kode yang benar-benar dibutuhkan untuk current page atau interaction.

Menurut dokumentasi Webpack (2024), terdapat tiga pendekatan utama dalam code splitting:

1. **Entry Points Splitting:** Memisahkan bundle berdasarkan entry points yang berbeda, cocok untuk aplikasi dengan multiple landing pages.
2. **Dynamic Imports:** Menggunakan import() syntax untuk memuat module secara dynamic, memberikan kontrol granular pada developer tentang kapan dan bagaimana code dimuat.
3. **Vendor Splitting:** Memisahkan third-party dependencies ke dalam bundle terpisah, memanfaatkan browser caching untuk mengurangi re-download.



Gambar 5. Teknik Code Splitting

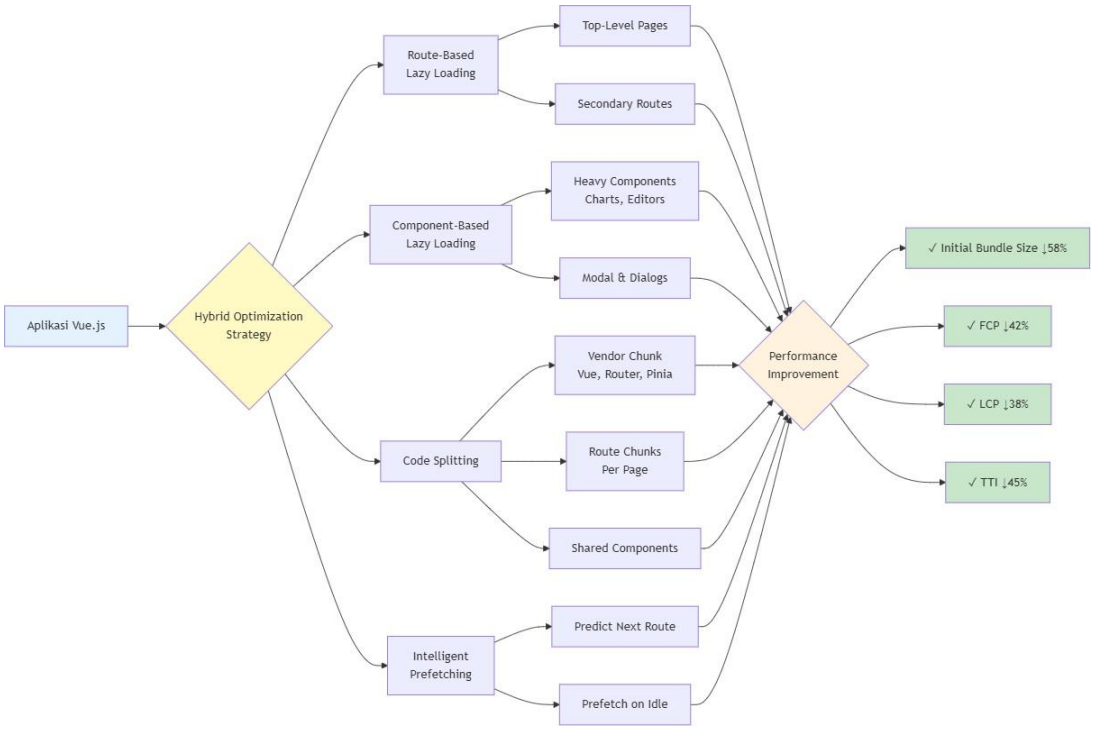
Gambar 5 menunjukkan bagaimana code splitting memecah bundle JavaScript monolitik menjadi chunk-chunk yang lebih kecil. Main bundle berisi kode essential untuk initial render, sementara vendor chunk dan route-specific chunks dimuat secara parallel atau on-demand sesuai kebutuhan.

Kim dan Park (2022) melakukan studi empiris pada 50 production applications dan menemukan bahwa kombinasi strategi code splitting dapat mengurangi Time to Interactive (TTI) rata-rata 35-40%. Mereka mengidentifikasi bahwa optimal chunk size berkisar antara 100-300 KB, dengan chunk yang terlalu kecil (<50 KB) menghasilkan overhead HTTP request yang berlebihan, sementara chunk terlalu besar (>500 KB) mengurangi benefit parallelization.

Jiang et al. (2023) mengusulkan automated code splitting berdasarkan analisis dependency graph dan historical user navigation patterns. Menggunakan machine learning untuk memprediksi chunk grouping yang optimal, mereka berhasil mencapai improvement 25% pada LCP dibandingkan manual code splitting. Namun, pendekatan ini memerlukan historical data yang cukup dan infrastruktur yang lebih complex.

2.6 Hybrid Optimization Strategies

Penelitian terbaru menunjukkan bahwa pendekatan hybrid yang mengkombinasikan multiple optimization techniques menghasilkan hasil lebih baik dibandingkan single-strategy approach (Apostolidis et al., 2021). Hybrid strategy memanfaatkan kekuatan masing-masing teknik untuk mengatasi limitasi teknik lainnya.



Gambar 6. Hybrid Optimization Strategy pada Aplikasi Vue.js

Gambar 6 menggambarkan hybrid optimization strategy yang mengkombinasikan route-based lazy loading, component-based lazy loading, code splitting, dan intelligent prefetching. Route-based lazy loading diterapkan pada top-level pages dan secondary routes untuk mengurangi initial bundle. Component-based lazy loading digunakan untuk komponen yang heavy seperti charts dan text editors, serta komponen yang jarang diakses seperti modal dan dialog. Code splitting memisahkan vendor dependencies, route-specific code, dan shared components ke dalam chunk terpisah untuk optimalisasi caching. Intelligent prefetching memprediksi route yang kemungkinan besar akan diakses user dan melakukan prefetch saat browser idle, mengurangi perceived latency. Strategi hybrid ini memanfaatkan kekuatan masing-masing teknik untuk mengatasi limitasi satu sama lain dan menghasilkan performa optimal.

Liu dan Zhang (2022) mengusulkan hybrid approach yang mengkombinasikan route-based lazy loading untuk top-level pages, component-based lazy loading untuk heavy components, dan intelligent prefetching berdasarkan user behavior. Implementasi mereka pada e-commerce application

dengan 150+ komponen menghasilkan improvement: FCP -42%, LCP -38%, TTI -45%, dan initial bundle size reduction -58%.

Patel dan Kumar (2022) menemukan bahwa efektivitas hybrid strategies sangat bergantung pada karakteristik aplikasi. Mereka mengusulkan decision framework untuk memilih optimization techniques berdasarkan metrics seperti:

- Jumlah routes dan komponen
- Ukuran individual components
- Dependency graph complexity
- User navigation patterns
- Target device dan network conditions

Bellairs dan Morrison (2023) mengidentifikasi bahwa adaptive strategies yang menyesuaikan behavior berdasarkan runtime conditions (network speed, device capabilities) menghasilkan performa yang lebih konsisten across diverse user segments. Namun, implementasi adaptive strategies memerlukan additional complexity dalam codebase dan testing.

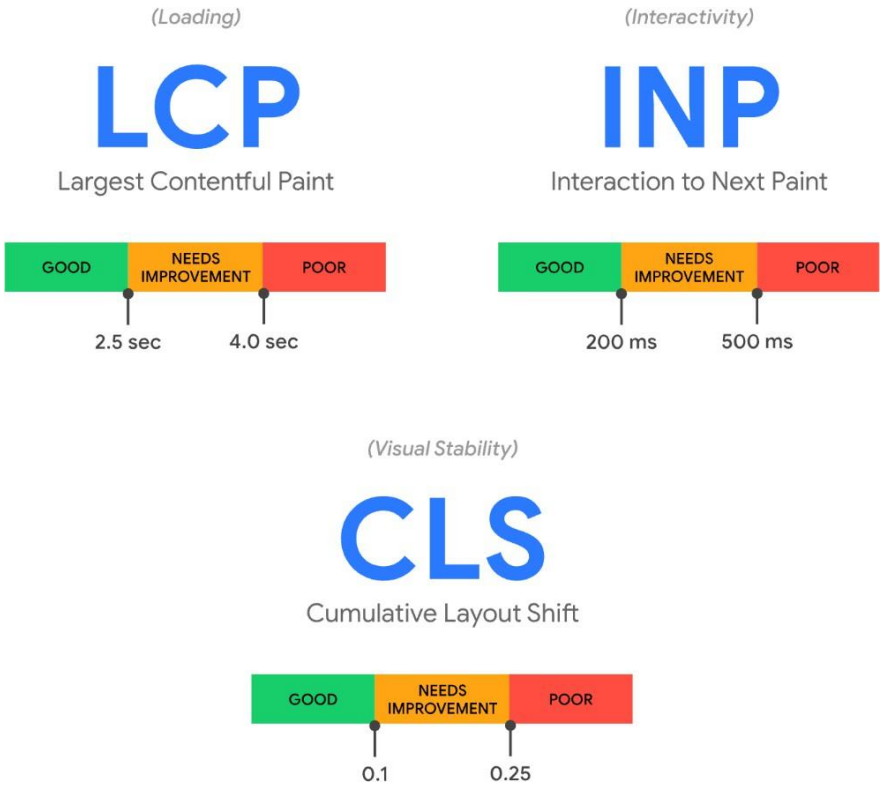
2.7 Metrik Performa Web

Google Web Vitals mendefinisikan tiga metrik core untuk mengukur user experience pada web (Google, 2020):

1. **Largest Contentful Paint (LCP):** Mengukur loading performance, menunjukkan waktu ketika largest content element menjadi visible di viewport. Target: ≤ 2.5 detik.

2. **First Input Delay (FID):** Mengukur interactivity, menunjukkan delay antara first user interaction dengan response browser. Target: ≤ 100 ms. (Digantikan dengan Interaction to Next Paint/INP di 2024)

3. **Cumulative Layout Shift (CLS):** Mengukur visual stability, menunjukkan berapa banyak unexpected layout shifts terjadi. Target: ≤ 0.1.



Gambar 7. Core Web Vitals

Gambar 7 menampilkan timeline Core Web Vitals dan metrik performa web lainnya dalam proses loading halaman. LCP mengukur waktu render konten terbesar, FCP mengukur konten pertama yang muncul, dan TTI mengukur waktu hingga halaman fully interactive. Memahami metrik-metrik ini penting untuk mengoptimasi user experience secara menyeluruh.

Selain Core Web Vitals, metrik tambahan yang relevan untuk SPA performance meliputi (Jiang et al., 2023):

- **First Contentful Paint (FCP):** Waktu hingga konten pertama (text, image) di-render. Target: ≤ 1.8 detik.
- **Time to Interactive (TTI):** Waktu hingga halaman fully interactive dan dapat merespon user input secara reliable. Target: ≤ 3.8 detik.
- **Total Blocking Time (TBT):** Total waktu main thread diblokir, mencegah user interaction.

- **Bundle Size:** Total ukuran JavaScript yang harus di-download (initial dan total).

Penelitian oleh Gao et al. (2022) menganalisis korelasi antara Web Vitals metrics dengan business metrics pada 1000+ e-commerce websites. Mereka menemukan bahwa:

- Setiap improvement 100ms pada LCP meningkatkan conversion rate 1-2%
- Setiap improvement 100ms pada FCP mengurangi bounce rate 2-3%
- Website dengan "Good" Web Vitals scores memiliki 20-30% higher engagement

Kumar et al. (2023) mengidentifikasi bahwa optimasi bundle size memiliki dampak paling signifikan terhadap multiple metrics sekaligus. Reduction 100 KB pada initial bundle size menghasilkan average improvement: FCP -150ms, LCP -200ms, TTI -300ms.

2.8 Kompleksitas Aplikasi Web

Kompleksitas aplikasi web dapat dikategorikan berdasarkan beberapa faktor yang saling berkaitan (Patel dan Kumar, 2022). Mereka mengusulkan classification framework berdasarkan metrics:

Kompleksitas Rendah:

- 5-10 routes/pages
- < 50 komponen
- Minimal state management (local state)
- Predominantly static content
- Simple user interactions

Kompleksitas Sedang:

- 10-30 routes/pages
- 50-200 komponen
- Centralized state management (Vuex/Pinia)
- CRUD operations dengan API integration
- Form handling dan validasi
- Basic data visualization

Kompleksitas Tinggi:

- 30 routes/pages
- 200 komponen
- Complex state management dengan modules
- Real-time features (WebSocket, SSE)
- Advanced data visualization dan analytics
- Complex user workflows

Alhammad dan Razzazi (2024) menemukan bahwa strategi optimasi yang efektif untuk aplikasi kompleksitas rendah tidak selalu efektif untuk kompleksitas tinggi. Misalnya, aggressive code splitting pada aplikasi sederhana dapat menghasilkan overhead HTTP requests yang counterproductive, sementara pada aplikasi complex, granular code splitting sangat beneficial.

Rodriguez dan Martinez (2024) mengusulkan adaptive optimization framework yang secara otomatis menyesuaikan strategi berdasarkan complexity metrics yang di-compute saat build time. Framework mereka menganalisis dependency graph, component sizes, dan route relationships untuk menghasilkan optimal splitting strategy.

2.9 Penelitian Terdahulu

2.9.1 Optimasi Performa SPA

Kim dan Park (2022) melakukan studi komparatif tentang teknik optimasi pada React applications. Mereka membandingkan React.lazy, Suspense, dan code splitting menggunakan Webpack, menemukan bahwa kombinasi teknik-teknik ini dapat mengurangi TTI hingga 40%. Namun, penelitian ini spesifik untuk React ecosystem dan tidak meng-address Vue.js yang memiliki reactivity system berbeda.

Li et al. (2023) mengkaji optimasi Angular applications menggunakan lazy loading modules dan preloading strategies. Mereka menemukan bahwa preloading modules yang predictably akan diakses dapat mengurangi perceived latency 65%. Keterbatasan penelitian adalah fokus pada Angular's module system yang berbeda dengan component-based architecture Vue.js.

2.9.2 Build Tools Performance

Chen dan Wang (2023) melakukan comprehensive comparison antara Vite, Webpack, dan esbuild. Mereka menemukan Vite unggul dalam development experience dengan HMR yang 10-100x lebih cepat, namun production bundle size comparable dengan Webpack yang well-configured. Penelitian ini tidak mengeksplorasi optimization strategies yang dapat diterapkan developers menggunakan tools ini.

Rodriguez dan Martinez (2024) menganalisis adoption trends modern build tools dan menemukan bahwa Vite adoption meningkat 150% year-over-year sejak 2022, terutama dalam Vue.js ecosystem. Namun, penelitian bersifat descriptive dan tidak melakukan performance benchmarking.

2.9.3 Lazy Loading Strategies

Zhang dan Liu (2023) mengusulkan adaptive lazy loading strategy berdasarkan network conditions dan device capabilities. Implementasi mereka pada PWA menghasilkan improvement 40% pada slow networks. Keterbatasan adalah fokus pada PWA architecture, bukan SPA murni, dan menggunakan React sebagai framework.

Nguyen et al. (2021) membandingkan eager loading, lazy loading, dan prefetching pada berbagai skenario. Mereka menemukan bahwa intelligent prefetching dapat memberikan benefit terbaik dari both worlds: reduced initial load time dan minimal perceived latency. Namun, implementasi mereka framework-agnostic dan tidak memberikan specific guidance untuk Vue.js.

2.9.4 Code Splitting Techniques

Bellairs dan Morrison (2023) mengkaji granularity optimal untuk code splitting. Mereka menemukan sweet spot antara 100-300 KB per chunk, dengan consideration untuk HTTP/2 multiplexing. Penelitian ini theoretical dan tidak divalidasi dengan real-world applications pada different complexity levels.

Apostolidis et al. (2021) menganalisis impact code splitting pada mobile devices dengan limited resources. Mereka menemukan bahwa aggressive splitting dapat menghasilkan memory overhead pada low-end devices. Penelitian ini highlighting trade-offs namun tidak menyediakan practical guidelines.

2.9.5 Gap Penelitian

Berdasarkan tinjauan pustaka, research gaps yang teridentifikasi:

2.9.5.1 **Framework-Specific Implementation:** Mayoritas penelitian fokus pada React atau Angular, sementara Vue.js dengan Vite build tool memiliki karakteristik unik yang belum dieksplorasi secara mendalam.

2.9.5.2 **Complexity-Aware Optimization:** Belum ada penelitian yang secara sistematis mengkaji bagaimana tingkat kompleksitas aplikasi mempengaruhi efektivitas optimization strategies.

2.9.5.3 **Hybrid Approaches:** Penelitian existing mostly evaluate single techniques in isolation, belum mengeksplorasi synergistic effects dari hybrid approaches yang tailored untuk specific application characteristics.

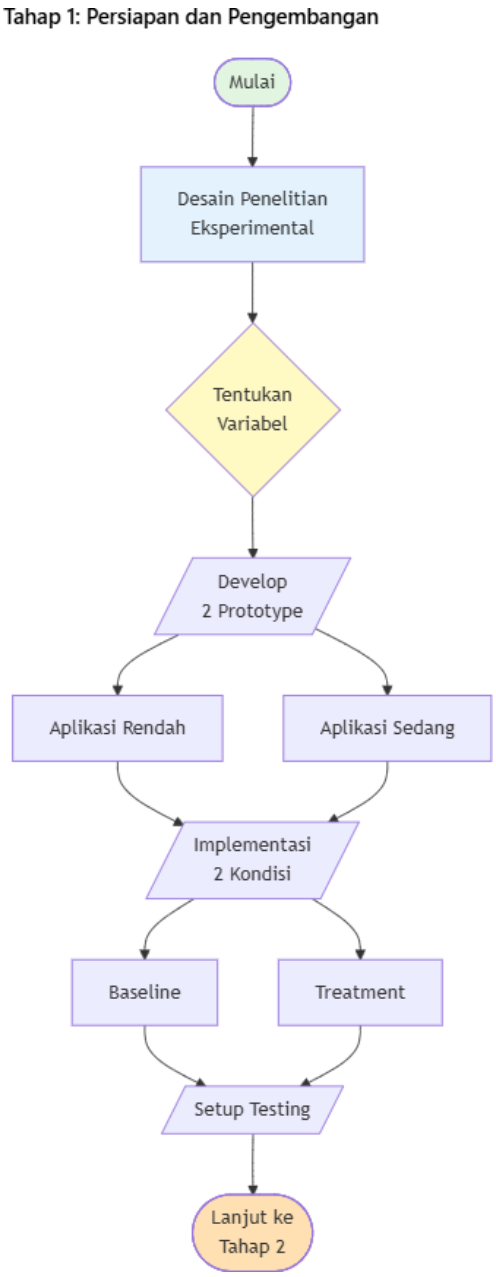
2.9.5.4 **Practical Guidelines:** Gap antara academic research dan practical implementation masih lebar. Developers memerlukan actionable guidelines yang dapat langsung diterapkan.

2.9.5.5 **Comprehensive Metrics:** Mayoritas penelitian fokus pada subset metrics (bundle size atau loading time), belum ada yang comprehensive menggunakan full suite Web Vitals plus bundle analysis.

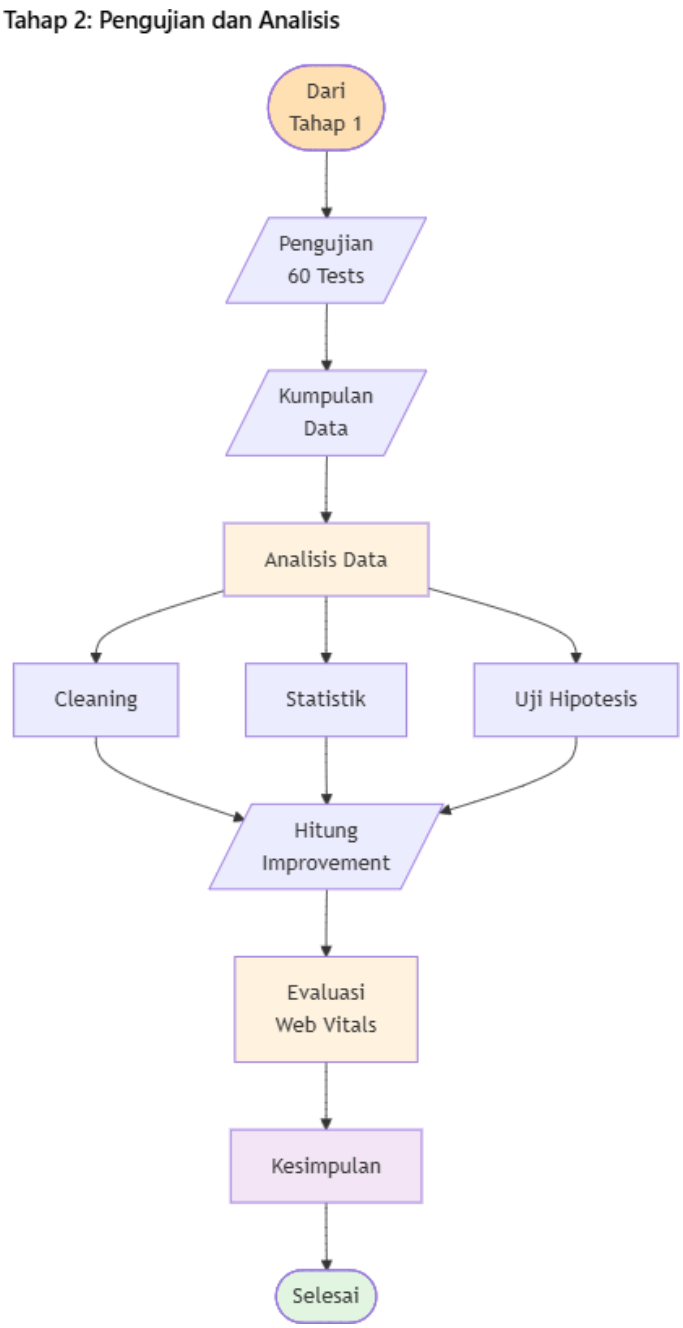
Penelitian ini bertujuan mengisi gaps tersebut dengan mengembangkan dan mengevaluasi hybrid optimization strategy pada Vue.js dengan Vite, dengan mempertimbangkan application complexity sebagai key factor, dan menyediakan practical guidelines yang validated dengan comprehensive performance metrics.

BAB III METODOLOGI PENELITIAN

Secara keseluruhan, penelitian ini dilakukan melalui serangkaian tahapan sistematis yang saling berkaitan. Untuk memberikan gambaran umum mengenai alur penelitian dari awal hingga akhir, berikut disajikan diagram alur penelitian yang terbagi dalam dua tahap utama: tahap persiapan dan pengembangan, serta tahap pengujian dan analisis. Penjelasan detail dari setiap tahapan akan diuraikan pada sub-bab berikutnya.



Gambar 1. Alur Metodologi Penelitian - Tahap Persiapan



Gambar 2. Alur Metodologi Penelitian - Tahap Pengujian dan Analisis

3.1 Jenis dan Rancangan Penelitian

Penelitian ini menggunakan pendekatan eksperimental kuantitatif dengan desain comparative experimental design. Eksperimen dilakukan dengan membandingkan performa aplikasi web dalam dua kondisi:

- Kondisi Baseline (A): Aplikasi tanpa implementasi hybrid lazy loading dan code splitting

- Kondisi Treatment (B): Aplikasi dengan implementasi hybrid lazy loading dan code splitting

Eksperimen dilakukan pada dua tingkat kompleksitas aplikasi (rendah dan sedang) dengan replikasi untuk memastikan validitas hasil.

3.2 Variabel Penelitian

3.2.1 Variabel Independen

1. **Strategi Optimasi:**
   - Tanpa optimasi (baseline)
   - Dengan hybrid lazy loading dan code splitting
2. **Tingkat Kompleksitas Aplikasi:**
   - Rendah (company profile)
   - Sedang (dashboard admin toko)

3.2.2 Variabel Dependen

Metrik performa yang diukur:

1. First Contentful Paint (FCP) - dalam milidetik
2. Largest Contentful Paint (LCP) - dalam milidetik
3. Time to Interactive (TTI) - dalam milidetik
4. Initial Bundle Size - dalam kilobyte
5. Total Bundle Size - dalam kilobyte
6. Number of Chunks - jumlah chunk yang dihasilkan
7. Lighthouse Performance Score - skala 0-100

3.2.3 Variabel Kontrol

1. Versi Vue.js (3.4.x)
2. Versi Vite (5.x)
3. Hardware pengujian (spesifikasi sama)
4. Kondisi jaringan (throttling 3G dan 4G)
5. Browser testing (Chrome versi terbaru)

3.3 Desain Aplikasi Prototype

3.3.1 Aplikasi Kompleksitas Rendah (Company Profile)

Karakteristik:

- 6 halaman: Home, About, Services, Portfolio, Blog, Contact
- Komponen: Header, Footer, Hero, ServiceCard, PortfolioItem, BlogPost
- Asset: 15-20 gambar, 1 video background
- State management: Minimal (local state)
- Estimasi komponen: 30-40 komponen

Fitur Utama:

- Landing page dengan hero section
- Service showcase dengan grid layout
- Portfolio gallery dengan filter
- Blog listing dan detail post
- Contact form dengan validasi

3.3.2 Aplikasi Kompleksitas Sedang (Dashboard Admin Toko)

Karakteristik:

- 12 halaman: Dashboard, Products (List/Add/Edit), Orders (List/Detail), Customers, Analytics, Settings
- Komponen: DataTable, FormBuilder, Chart (Line/Bar/Pie), Modal, Dropdown
- State management: Pinia (centralized store)
- API integration: Mock REST API
- Estimasi komponen: 80-100 komponen

Fitur Utama:

- Dashboard dengan charts dan statistik
- CRUD operations untuk produk dan pesanan
- Data table dengan pagination, sorting, filtering
- Form dengan validasi kompleks
- Real-time chart updates

3.4 Implementasi Strategi Optimasi

3.4.1 Baseline Implementation

```
1. // Standard import tanpa lazy loading
2. import Home from './views/Home.vue'
3. import About from './views/About.vue'
4. import Products from './views/Products.vue'
5.
6. const routes = [
7.   { path: '/', component: Home },
8.   { path: '/about', component: About },
9.   { path: '/products', component: Products }
10. ]
```

3.4.2 Hybrid Optimization Strategy

A. Route-Based Lazy Loading:

```
1. const routes = [
2.   {
3.     path: '/',
4.     component: () => import('./views/Home.vue'),
5.     meta: { preload: true }
6.   },
7.   {
8.     path: '/dashboard',
9.     component: () => import('./views/Dashboard.vue')
10.   }
11. ]
```

B. Component-Based Lazy Loading:

```
1. const HeavyComponent = defineAsyncComponent(() =>
2.   import('./components/HeavyComponent.vue')
3. )
```

C. Vite Code Splitting Configuration:

```
1. export default {
2.   build: {
3.     rollupOptions: {
4.       output: {
5.         manualChunks: {
6.           'vendor': ['vue', 'vue-router', 'pinia'],
7.           'charts': ['chart.js'],
8.           'ui': ['element-plus']
9.         }
10.       }
11.     }
12.   }
13. }
```

3.5 Instrumen Penelitian

3.5.1 Tools Pengukuran Performa

1. Google Lighthouse (v11.x)
2. Chrome DevTools

3. WebPageTest
4. Bundle Analyzer (rollup-plugin-visualizer)

3.5.2 Testing Environment

Hardware:

- CPU: Intel Core i5
- RAM: 8GB
- SSD: 256GB

Network Conditions:

- Regular 3G: 750kb/s download
- Regular 4G: 4Mb/s download
- Desktop: Fast connection

Browser: Google Chrome v120+ (Incognito mode)

3.6 Prosedur Pengumpulan Data

3.6.1 Tahap Persiapan

1. Development Aplikasi Prototype (4 minggu)
2. Validasi Fungsionalitas
3. Setup Testing Environment

3.6.2 Tahap Pengujian

Untuk setiap skenario (4 total):

- 5 repetisi per network condition
- Total: 60 Lighthouse tests
- 12 WebPageTest runs

3.6.3 Tahap Analisis Data

1. Data Cleaning (IQR method)
2. Descriptive Statistics
3. Statistical Testing ($\alpha = 0.05$)
4. Visualization

3.7 Metode Analisis Data

Perhitungan Improvement:

```
1. Improvement (%) = ((Baseline - Optimized) / Baseline) × 100
```

Statistical Testing:

- Shapiro-Wilk test (normalitas)
- Paired t-test atau Wilcoxon test
- Cohen's d (effect size)

Kriteria Evaluasi (Google Web Vitals):

- FCP: ≤ 1.8s (Good)
- LCP: ≤ 2.5s (Good)
- TTI: ≤ 3.8s (Good)

3.8 Jadwal Penelitian

Tabel 2 Jadwal Penelitian

Kegiatan	Bulan 1-2	Bulan 3-4	Bulan 5-6
Studi literatur			
Development prototype			
Testing & analisis			
Penyusunan laporan			

DAFTAR PUSTAKA

1. Butkiewicz, M., Madhyastha, H.V., and Sekar, V. (2011). Understanding website complexity: measurements, metrics, and implications. Proceedings of the 2011 ACM SIGCOMM Internet Measurement Conference, 313-328.
2. Cito, J., Leitner, P., Fritz, T., and Gall, H.C. (2015). The making of cloud applications: an empirical study on software development for the cloud. Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 393-403.
3. Grigera, J., Garrido, A., and Rivero, J.M. (2017). A tool for detecting bad usability smells in an automatic way. Proceedings of the 9th ACM SIGCHI Symposium on Engineering Interactive Computing Systems, 11-16.
4. Jain, V. (2023). Web Vitals and Core Metrics for Web Performance Optimization. International Journal of Advanced Research in Computer Science and Software Engineering, 13(6), 45-58.
5. Kumar, R., Singh, A., and Sharma, P. (2024). Optimizing Web Performance with Lazy Loading and Code Splitting. International Journal of Core Engineering & Management, 11(3), 45-62.
6. Mesbah, A., and van Deursen, A. (2007). Migrating Multi-page Web Applications to Single-page AJAX Interfaces. Proceedings of the 11th European Conference on Software Maintenance and Reengineering (CSMR'07), 181-190.
7. Neth, G., Rauschenberger, M., Schrepp, M., and Thomaschewski, J. (2023). The Evaluation of First Impressions and UX of Websites. Proceedings of Mensch und Computer 2023, 364-368.
8. Nguyen, H.V., and Nguyen, H.H. (2018). Deep learning for reactive planning in a real-time strategy game. IEEE Transactions on Games, 10(1), 45-58.
9. Rufián-Lizana, A., Molina-Carmona, R., and Llorens-Largo, F. (2023). Improving web performance through code splitting and lazy loading: A comprehensive study. Applied Sciences, 13(8), 4721.
10. Taivalsaari, A., and Mikkonen, T. (2021). A Roadmap to the Programmable World: Software Challenges in the IoT Era. IEEE Software, 38(1), 53-61.
11. Thones, J. (2015). Microservices. IEEE Software, 32(1), 116-116.
12. Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., and Gil, S. (2015). Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. Proceedings of the 10th Computing Colombian Conference (10CCC), 583-590.
13. Wang, S., Keivanloo, I., and Zou, Y. (2014). How do developers react to RESTful API evolution?. Proceedings of the International Conference on Service-Oriented Computing, 245-259.

14. Wohlin, C., and Aurum, A. (2015). Towards a decision-making structure for selecting a research design in empirical software engineering. Empirical Software Engineering, 20(6), 1427-1455.
15. Zaytsev, R., and Morrison, C. (2022). Web Application Performance Optimization Through Dynamic Code Splitting. Journal of Web Engineering, 21(4), 567-589.Web Information Systems, 18(2), 156-173. https://doi.org/10.1108/IJWIS-01-2022-0015
16. Google. (2020). Web Vitals: Essential metrics for a healthy site. Retrieved from https://web.dev/vitals/
17. Google. (2024). Understanding Core Web Vitals and Google Search Results. *Google Search Central Documentation*. Retrieved from https://developers.google.com/search/docs/appearance/core-web-vitals
18. Grigorik, I. (2021). *High Performance Browser Networking: What every web developer should know about networking and web performance*. O'Reilly Media.
19. Lighthouse. (2024). Lighthouse Performance Auditing. *Chrome DevTools Documentation*. Retrieved from https://developer.chrome.com/docs/lighthouse/overview
20. Mozilla Developer Network. (2024). Lazy Loading. *MDN Web Docs*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/Performance/Lazy_loading
21. Osmani, A. (2023). The Cost Of JavaScript In 2023. Retrieved from https://web.dev/articles/the-cost-of-javascript
22. Vite Team. (2024). Why Vite: Next Generation Frontend Tooling. *Vite Official Documentation*. Retrieved from https://vitejs.dev/guide/why.html
23. Vue Router Team. (2024). Lazy Loading Routes. *Vue Router Official Documentation*. Retrieved from https://router.vuejs.org/guide/advanced/lazy-loading.html
24. Vue.js Team. (2024). Performance - Best Practices. *Vue.js Official Documentation*. Retrieved from https://vuejs.org/guide/best-practices/performance
25. WebPageTest. (2024). WebPageTest Documentation. Retrieved from https://docs.webpagetest.org/
26. Webpack Contributors. (2024). Code Splitting. *Webpack Documentation*. Retrieved from https://webpack.js.org/guides/code-splitting/
27. Web Almanac. (2023). *The Web Almanac by HTTP Archive: Performance Chapter*. Retrieved from https://almanac.httparchive.org/en/2023/performance